PROJECT DOCUMENTATION

# Stackly Insurance Service

## Technical Documentation

A complete reference for the Stackly Insurance Service website. It documents the project structure, design tokens, per-page breakdown, responsive behaviour, client-side data layer, and GitHub deployment. Written for developers and reviewers who need to understand, extend, or maintain the project.

# Table of Contents

*All section references are to page numbers in this document.*

1 Introduction 3

2 Product Overview & System Architecture 4

3 Color Palette 5

4 Typography & Design Tokens 6

5 Project Structure 7

6 Home Page (index.html) 8

7 About & Products Pages 9

8 Claims & Resources Pages 10

9 Contact & 404 Pages 11

10 Auth Pages (Login / Sign Up) 12

11 Customer Dashboard 13

12 Agent Dashboard 14

13 Data & State Management 15

14 Responsiveness 16

15 Deployment (GitHub Pages) 17

16 Conclusion 18

# 1 Introduction

## 1.1 Purpose

This document is the technical reference for the Stackly Insurance Service website. It describes what the project is, how it is put together, what each page does, and how it is styled, made responsive, and deployed. The intent is to give a developer who is new to the project enough context to work on it safely, and to give a reviewer a single place to check conventions.

## 1.2 Scope

The project is a front-end-only website. There is no server, no database, and no build step. All content, styling, and behaviour live in plain HTML, CSS, and JavaScript inside the project folder. The document covers:

- the eleven HTML entry pages, including two role-based dashboard applications;
- the shared design system (colors, typography, spacing and motion tokens);
- the client-side data layer that simulates an account and a backend;
- responsive behaviour down to 320 px; and
- hosting the finished site on GitHub Pages.

## 1.3 Audience

The reader is assumed to be comfortable with semantic HTML5, CSS custom properties, and vanilla JavaScript. No framework knowledge is required because the project deliberately avoids frameworks.

## 1.4 Document Conventions

- File paths are written relative to the project root, for example `assets/css/style.css`.
- Color and text values are referenced by their token names where relevant, for example `--navy`, defined on the `:root` selector.
- Every behavior script is an immediately-invoked function expression (IIFE) in strict mode, so nothing leaks into the global scope.
- On the dashboard pages, account and session data is kept in the browser's `localStorage`; these keys are listed in section 13.

## 1.5 Delivered Inventory

| Item | Count | Notes |
|---|---|---|
| HTML entry pages | 11 | Six marketing pages, contact, 404, two auth pages, two dashboards |
| Stylesheets | 10 | One shared base sheet, one per page, one responsive pass |
| JavaScript files | 13 | One shared shell script, one per page, one shared data store |
| Image assets | 133 | All WebP; the largest file is about 86 KB |
| Fonts | 2 | Manrope (display) and DM Sans (body), loaded from Google Fonts |

# 2 Product Overview & System Architecture

## 2.1 Product Summary

Stackly Insurance Service is a premium insurance website that presents cover for Home, Auto, Health, Life, Travel, and Business. The site is built around a warm, human tone (the tagline on most pages is “Good cover for the life you are building”). It serves two audiences: visitors who browse products, read resources, and make contact, and customers or agents who sign up and then work inside a dashboard.

The experience is split into three logical parts. The marketing pages explain the products and the company. The auth pages create and sign in accounts. The dashboards then render that account’s workspace, one designed for a customer and one for an insurance agent.

## 2.2 Architecture Notes

- **Static by design.** The site runs without a server. Serving the folder over HTTP is enough (any static host works).
- **Shared shell, duplicated markup.** The site header, announcement bar, and footer are the same HTML on the marketing pages. They are repeated rather than injected so pages work without JavaScript.
- **Layer 1 - presentation.** `assets/css/style.css` holds all tokens, the shell, and shared components. Each inner page adds a namespaced sheet, and `responsive.css` (loaded last) applies the shared typography and small-screen passes.
- **Layer 2 - behaviour.** `assets/js/main.js` powers the shared shell (sticky header, mobile menu, scroll reveals, FAQ accordions, forms). Each page adds its own script for page-specific behaviour.
- **Layer 3 - data.** `assets/js/store.js` is the single source of truth. Both dashboards and both auth pages read and write account, session, and demo data through this module, which stores everything in `localStorage`.
- **Dashboards are hash-routed SPAs.** Customer and agent dashboards are single-page applications rendered by JavaScript. Views map to URL hashes such as `#/policies`; switching a hash swaps the routed view without a page reload.

## 2.3 Key Technologies

| Layer | Technology | Where / Why |
|---|---|---|
| Markup | Semantic HTML5 | Header, nav, main, section, article, figure landmarks and ARIA roles throughout |
| Styling | Hand-authored CSS, custom properties | No framework; tokens on `:root`, per-page namespaced sheets |
| Behaviour | Vanilla JavaScript (IIFE) | No build step, no framework; real browser APIs only |
| Charts | ApexCharts, Chart.js fallback | Loaded lazily from CDN inside the dashboards; inline SVG as the last fallback |
| Data | `localStorage` | Simulated account/session layer; deterministic demo seeding |
| Maps | OpenStreetMap embed | Keyless iframe on the contact page, centered on the Salem office |
| Typography | Google Fonts | Manrope and DM Sans via preconnected stylesheet link |

# 3 Color Palette

All colors are defined once on the `:root` selector in `assets/css/style.css` and referenced through their token names. There are twelve tokens. The palette is built around deep green-navy tones with a warm cream canvas and a soft mint accent, which together give the brand its calm, human feel.

| Swatch | Token | Hex | Role | Primary usage |
|---|---|---|---|---|
| | `--navy` | #0d2528 | Primary dark | Hero sections, footer, dark buttons, auth brand panel |
| | `--navy-deep` | #071719 | Deepest tone | Overlay gradients, image washes |
| | `--blue` | #1e4a4a | Secondary dark | Inner section bands, closing CTA panels, category cards |
| | `--blue-soft` | #356867 | Accent teal | *em* emphasis inside large headings |
| | `--mint` | #b7e6c8 | Light accent | Highlight labels, mint CTA buttons, success marks |
| | `--mint-deep` | #4d9b76 | Brand accent | Links, eyebrows, focus rings, validation success |
| | `--cream` | #f5f1e8 | Page background | Body surface of the marketing pages |
| | `--sand` | #e4ded1 | Alternating band | Contact map/office and pricing band backgrounds |
| | `--paper` | #ffffff | Card surface | Cards, tables, auth and dashboard panels |
| | `--ink` | #183133 | Body text | Primary copy color on light surfaces |
| | `--muted` | #6b7a78 | Secondary text | Descriptions, captions, placeholders |
| | `--line` | #d9ddd5 | Border / divider | Inputs, table rows, card borders |

## 3.1 Usage Conventions

- Dark sections (hero, footer, closing CTA) sit on `--navy`; light headings on those surfaces are white with a mint *em*.
- Large section headings on light backgrounds emphasise one word with `--blue-soft`.
- Interactive focus is always the `--mint-deep` outline; form errors are a fixed red (`#a33d46`) used only in error states.
- The dashboard theme toggle switches between the light palette and a dark variant that keeps the same accent tokens.

# 4 Typography & Design Tokens

## 4.1 Font Families

Two families are loaded from Google Fonts with preconnect hints. The display face is **Manrope** (weights 500-800) and is used for all headings and buttons. The body face is **DM Sans** (weights 400-700) and is used for copy, labels, and inputs. The pair is used consistently across every page, including the dashboards.

| Token | Family | Purpose |
|---|---|---|
| `--display` | Manrope, Arial, sans-serif | h1-h3, buttons, numbers, strong labels |
| `--body` | DM Sans, Arial, sans-serif | Paragraph text, forms, small print |

## 4.2 Heading Scale

Headings are fluid: they use `clamp()` so they scale smoothly with the viewport. The shared scale used by every inner page is defined in `responsive.css` under `body[class] main h1/h2`.

| Element | Rule | Smallest / Largest |
|---|---|---|
| h1 | `clamp(3.25rem, 5.7vw, 6rem)` | 52px / 96px |
| h2 | `clamp(2.65rem, 4.3vw, 4.45rem)` | 42px / 71px |
| h1 (mobile) | `clamp(3.2rem, 14.5vw, 5.4rem)` | down to ~41px at 320px viewports |

## 4.3 Spacing, Radius & Shadow Tokens

- Page container: `width: min(1240px, calc(100% - 64px))`, centered. The gutter shrinks as the viewport narrows.
- Major section padding is generally `128px - 145px` vertical, tightened by media queries at smaller widths.
- Radii: `--radius-sm: 10px`, `--radius-md: 18px`, `--radius-lg: 28px`, `--radius-pill: 999px`. Cards alternate between straight and generously rounded corners (e.g. `24px 170px 24px 24px` on the map frame).
- Shadows: `--shadow-soft` and `--shadow-card` give depth to overlapping media and dashboard cards.

## 4.4 Imagery & File Standards

- All photographs are WebP, optimised, and kept below 100 KB (the current largest is about 86 KB). This keeps every page light.
- Photos come from Unsplash under the Unsplash License and live in `assets/images/` grouped by page (uncropped images used directly at the project root for legacy assets).
- Avatars in the dashboards fall back to brand-gradient initials when no photo is set, so no placeholder image is required.
- The brand logo must stay at the fixed path `assets/logo/logo-stackly.webp`; both header and footer reference it directly.

## 4.5 Motion

Scroll reveals use IntersectionObserver, trust metrics animate on first sight, and the 404 page has a cancelable 15-second redirect. Every animation respects `prefers-reduced-motion`; the dashboards also expose an in-app “Reduce motion” and “Larger text” preference.

# 5 Project Structure

The project is a flat web root. Entry pages sit at the top level; all reusable assets are grouped under `assets/`. There is no configuration folder and nothing to compile.

```
index.html                  Landing / home page
about.html                  Company story, team, culture
products.html               Product catalogue and coverage builder
claims.html                 Claims process, tracker, stories
resources.html              Guides, articles, webinars
contact.html                Contact form, methods, Salem office map
404.html                    Not-found page with recovery helpers
login.html                  Sign-in (Customer / Agent)
signup.html                 Three-step account creation
customer-dashboard.html     Customer role-based SPA
agent-dashboard.html        Agent role-based SPA
assets/
  logo/logo-stackly.webp    Brand mark (path is fixed)
  images/
    about/  contact/  products/  claims/  resources/  404/
    auth/   dashboard/          page-specific photography
    hero-family-shield.webp ... shared homepage images
  css/
    style.css       tokens, shell, shared components
    about.css  products.css  claims.css  resources.css
    contact.css  404.css  auth.css  dashboard.css
    responsive.css  shared typography + small-screen pass
  js/
    main.js         shared shell behaviours
    about.js  products.js  claims.js  resources.js
    contact.js  404.js       page-specific behaviours
    auth.js  login.js  signup.js
    store.js        shared data layer (localStorage)
    dashboard.js    customer SPA + chart adapter
    agent.js        agent SPA logic
```

## 5.1 How a Page Is Assembled

1. The HTML page references `style.css` first, then its own sheet, then `responsive.css` last. Later sheets can therefore override shared defaults safely.
2. Behaviour scripts are loaded with `defer` at the end of `<body>`, in this order on the shared pages: `main.js`, then the page script.
3. Auth pages additionally load `store.js` before their page script so the data layer exists on first run.
4. The dashboards load `main.js`, `store.js`, then `dashboard.js` (shared shell and charts), and the agent dashboard adds `agent.js`.

## 5.2 Naming Convention

Page-specific CSS is namespaced: the products page uses `.products-`, claims uses `.claims-`, the 404 page uses `.nf-` / `.notfound-`, and the agent dashboard uses `.ag-`. This prevents selector collisions between pages while keeping each sheet readable. The exception is `dashboard.css`, which serves both dashboards and uses a `dash-` and `ov-` prefix set.

# 6 Home Page (index.html)

The landing page has no `body` class, so it runs on the base styles alone. Its layout order is enforced in CSS (`main > .hero-section` and so on) so the sections always appear in the intended order. It is the longest page on the site and demonstrates every shared component.

| # | Section | Purpose / Behaviour |
|---|---|---|
| 1 | Site header | Sticky header with scroll-shrink, scroll progress bar, announcement bar, desktop nav and slide-down mobile menu |
| 2 | Hero | Headline with mint *em*, floating proof badges (98.7% claims satisfaction), background grid and orbs |
| 3 | Trust strip | Animated counters (claims satisfaction, average decision time, people covered) that count up on reveal |
| 4 | Products | Six product cards (Home, Auto, Health, Life, Travel, Business) linking to products.html |
| 5 | Why Stackly | Feature blocks explaining the brand values with supporting photography |
| 6 | How it works | Three numbered steps that summarise the buying journey |
| 7 | Quote funnel | A client-side three-step teaser (cover type, details, contact) that resolves to a confirmation state |
| 8 | Claims experience | Banner selling the claims experience with human-support messaging |
| 9 | Testimonials | Keyboard-operable carousel (previous / next / dots / arrow keys) with three customer stories |
| 10 | Plans | Pricing cards for Essential, Premium, and Elite, with the Premium card featured |
| 11 | FAQ & final CTA | Single-open accordion plus a quote CTA and newsletter form with confirmation states |
| 12 | Footer | Brand column, product / company / resource links, newsletter, contact row, legal row |

> All shared behaviours used here (sticky header, reveals, metrics, carousel, accordion, forms, mobile menu) are owned by `main.js` and are reused by the inner pages automatically because they share the same header and footer markup.

# 7 About & Products Pages

## 7.1 about.html

The About page tells the company story. Its styles live in `assets/css/about.css` (`.about-` namespace) and its behaviour in `about.js`.

| # | Section | Purpose / Behaviour |
|---|---|---|
| 1 | About hero | Split hero with the “Started with a simple belief” headline and layered photography |
| 2 | Story timeline | Numbered milestones with scroll reveal |
| 3 | Mission, vision, values | Three-column values treatment; the third value is positioned as a dark panel |
| 4 | Leadership & team | Team cards with initials/photo avatars |
| 5 | By the numbers | Animated counters (people covered, claims, retention) |
| 6 | Culture & gallery | Office and workshop photography grid |
| 7 | Trust, community & CTA | Commitment badges (Licensed, Privacy by design, Community minded), closing CTA |

## 7.2 products.html

The Products page is the selling core of the site. Styles are in `assets/css/products.css` (`.products-`) and interaction logic in `products.js`.

| # | Section | Purpose / Behaviour |
|---|---|---|
| 1 | Products hero | Headline with a parallax “coverage board” collage and floating stats |
| 2 | Category navigator | Six image cards (Auto, Home, Life, Health, Travel, Business); the mint and dark cards vary the rhythm |
| 3 | Flagship spotlight | Home cover showcase with tabbed panels (Clear cover / Flexible choices / Human care) |
| 4 | Compare plans | Three-plan chip strip and a feature matrix table with a featured Premium column |
| 5 | Coverage builder | Interactive estimator: choose a product type, drag a level slider (1-3), reads a live price and description |
| 6 | Benefits & proof | Stats counters and a customer-story slider with controls |
| 7 | Pricing, FAQ & CTA | Three pricing cards, single-open FAQ accordion, and a rounded closing CTA band with photography |

> The coverage builder is client-side only. It maps product type + level to fixed example prices (for example home cover $38/$48/$82 per month) and is clearly labelled a guide, not a hard quote.

# 8 Claims & Resources Pages

## 8.1 claims.html

The Claims page explains the claims experience and includes a working tracker demo. Styles are in `assets/css/claims.css` (`.claims-`) and behaviour in `claims.js`.

| # | Section | Purpose / Behaviour |
|---|---|---|
| 1 | Claims hero | Statement hero with the claims-support agent imagery |
| 2 | How claims work | Four numbered process steps with reveal animation |
| 3 | Fast payout | Payout speed messaging with a progress-style visual |
| 4 | Claim types | Cards for Home, Auto, Health, Life, Travel, Business claim types |
| 5 | Track & manage | Claim tracker demo: entering any claim ID (e.g. STK-2048-17) shows a four-stage timeline with a “Details under review” active state |
| 6 | Real stories | Customer story slider with the same controls and accessible semantics as the homepage carousel |
| 7 | FAQ & closing CTA | Single-open accordion and rounded closing band |

## 8.2 resources.html

The Resources page is a content hub. Styles are in `assets/css/resources.css` (`.resources-`) and behaviour in `resources.js`, which also drives a video lightbox.

| # | Section | Purpose / Behaviour |
|---|---|---|
| 1 | Knowledge hub hero | Intro with a notebook visual |
| 2 | Featured resources | Two large feature cards |
| 3 | Browse by category | Category grid (Guides, Articles, Webinars, Case studies, Tools, Blog, Video) |
| 4 | Latest articles | Article card grid (Home, Business, Life, Health, Auto, Travel) |
| 5 | Guides & downloads | Download-style guide cards |
| 6 | Webinars & video library | Video cards; clicking opens a modal lightbox with the video embed |
| 7 | Newsletter, FAQ & CTA | Newsletter form, FAQ accordion, and closing band |

# 9 Contact & 404 Pages

## 9.1 contact.html

The Contact page routes visitors to the right channel and hosts the fully validated contact form. Styles are in `assets/css/contact.css` ( `.contact-` ) and behaviour in `contact.js`.

| # | Section | Purpose / Behaviour |
|---|---|---|
| 1 | Contact hero | Split hero with response-time proof strip (4 min average response, 24/7 claims) |
| 2 | Methods & form | Call / Email / Visit cards plus the contact form below |
| 3 | Map & Salem office | Lazy-loaded OpenStreetMap iframe (lat 11.6643, lon 78.1460), office hours, address card |
| 4 | FAQ & closing CTA | Single-open accordion and closing band |

### Contact form validation

The form is validated on submit, blur, change, and live input. Each field has its own inline `role="alert"` message. The rules are:

- **Name** - required, letters only (a-z plus accented letters, spaces, apostrophes, hyphens); invalid characters are stripped as you type; minimum 2 characters.
- **Email** - required, standard address pattern.
- **Phone** - optional; if entered it must contain 7-15 digits allowing only + ( ) - . and spaces.
- **Topic** - required; a selection must be made.
- **Message** - required, 12-2000 characters.

On success the form is hidden and replaced by a confirmation panel with a “Send another message” reset. Submission is simulated on the client; the API hook point is in `contact.js`.

## 9.2 404.html

The 404 page reuses the full marketing header and footer so visitors can recover through normal navigation. It is marked `noindex, nofollow` and styled by `assets/css/404.css`, scripted by `404.js`.

- **Hero** - floating 404 digits, a cancelable 15-second auto-redirect countdown exposed via `aria-live`, and a parallax illustration.
- **Go to Home / Go Back** - Home links to index.html; Go Back calls `window.history.back()` and falls back to index.html when there is no referrer.
- **Helpful quick links** - image cards linking to Products, Claims, Resources, Contact, and Account.
- **Recovery search** - a search field, plus the trust badges and contact row.

# 10 Auth Pages (Login / Sign Up)

Login and signup are deliberately lightweight. They share a minimal shell (logo, opposite-page link, slim legal footer) instead of the full marketing navigation, so signing in and onboarding stay focused. Styling is in `assets/css/auth.css`; validation logic is shared in `auth.js` and completed in `login.js` and `signup.js`.

## 10.1 Role selection

Both forms support exactly two roles: **Customer (Policyholder)** and **Agent**, chosen from a radio-group of two buttons. On login the selected role must match the account role; signup persists the role with the account. Agent users land on `agent-dashboard.html` and customers on `customer-dashboard.html`.

## 10.2 login.html

- Role radio group, email field, password field (minimum 8 characters).
- Validation runs on blur and submit with accessible inline messages.
- An unknown demo email logs in by creating a lightweight account (no password stored) so the dashboard is always reachable.
- Submission is simulated; the loading state resolves locally. Credentials are never logged or stored.

## 10.3 signup.html

Account creation is a three-step wizard with a progress bar at the top of the panel:

1. **Who you are** - role, full name, email.
2. **Contact & location** - phone, country, and (depending on role) the customer policy interest or the agent license / agency ID.
3. **Security & preferences** - password with complexity rule, matching confirmation, communication preferences, and required Terms acceptance.

Validation mirrors `auth.js` conventions: name, email and confirmation, phone, country, password strength, matching confirmation, role-specific fields, and Terms. A password strength meter and per-step error messages guide the user. On completion a success screen links to the correct dashboard.

## 10.4 Shared account data

The `Go to home` button sits in the auth header for visitors who change their mind. New accounts are persisted through `StacklyStore.createUser(...)` and sessions through `setSession(...)`, both from `store.js` (see section 13).

# 11 Customer Dashboard (customer-dashboard.html)

The customer dashboard is a hash-routed single-page application written in vanilla JavaScript. It reuses the shared local CSS and adds dashboard-specific styles from `assets/css/dashboard.css`. Behaviour is in `dashboard.js`, which exposes the shared chart adapter on `window.StacklyDashboard`.

## 11.1 Application shell

- **Sidebar** - persistent navigation with a collapse toggle, role badge, and user card.
- **Topbar** - page title, global search, notifications, user menu, and a light/dark theme toggle.
- **Content area** - the routed view renders here; scroll is contained within the shell.

## 11.2 Routes

| Hash route | View | Content |
|---|---|---|
| `#/overview` | Overview | Welcome card, KPI sparklines, coverage donut, premium area chart, claims stacked bar, protection radial gauge |
| `#/policies` | Policies | Policy cards (Home, Auto, Health, Life, Travel, Business), coverage bar, breakdown donut |
| `#/claims` | Claims | Claim list with statuses, status donut, claims-by-month chart, evidence upload mock |
| `#/payments` | Payments | Payment history, premium vs payout bar chart, status donut |
| `#/documents` | Documents | Document library with type donut and download-style rows |
| `#/support` | Support | Help topics, support tickets, response radial gauge, contact card |
| `#/profile` | Profile | Editable profile form, completion radial, account details |
| `#/settings` | Settings | Preferences: theme, email/SMS/claim alerts, reduce motion, larger text |

## 11.3 Charts

Charts use ApexCharts loaded lazily per session from `cdn.jsdelivr.net`. If that fails, the adapter falls back to Chart.js; if both fail, sparklines and gauges render as inline SVG and other charts show an accessible text fallback. Charts are created when a view mounts, disposed on route change, and every container carries a visual `role="img"` label plus a visually hidden summary.

> The dashboards are deliberately chart-heavy, so Lighthouse scores around 90+ rather than 95+ (documented tradeoff). This is acceptable for the deliverable because charts load only when needed.

# 12 Agent Dashboard (agent-dashboard.html)

The agent dashboard is the second role-based SPA. It reuses the exact shared shell from the customer dashboard (sidebar, topbar, drawer, toast, chart adapter) and adds agent-only logic in `agent.js` with an `.ag-` CSS namespace. Both dashboards can be open at the same time because sessions are stored per role.

## 12.1 Routes

| Hash route | View | Content |
|---|---|---|
| `#/overview` | Overview | Six KPI sparklines, premium-written area chart, policies-by-type donut, claims queue bar, leads funnel, commission bar, two target radial gauges |
| `#/customers` | Customers | Client table with status, premium-by-city bar, coverage donut, avatar initials fallback |
| `#/policies` | Policies | New-vs-renewals stacked bar, book-by-product donut, policy detail cards |
| `#/claims` | Claims queue | Status donut, monthly stacked bar, time-to-settle area, queue funnel, assignable claim cards |
| `#/leads` | Leads / pipeline | Pipeline bar, win-rate donut, kanban board with keyboard move controls |
| `#/commissions` | Commissions | Earnings bar chart and product-mix donut |
| `#/appointments` | Appointments | Meetings-per-week bar chart and schedule list |
| `#/messages` | Messages | Inbox thread list and conversation pane |
| `#/reports` | Reports | Revenue-vs-target bar, product-mix donut, premium-by-city bar, conversion area |
| `#/profile` / `#/settings` | Profile / settings | Same profile editing and preferences as the customer dashboard, including agency ID |

## 12.2 Agent workspace data

Agent data is stored separately from customer data, keyed by agent ID, and covers region, agency ID, monthly targets (premium, commission, policies, conversion rate), plus customers, policies, claims, leads, commissions, appointments, messages, notifications, and activity. Example records include a `score` and `tags` on customers and a sales `stage` / `value` on leads.

## 12.3 Shared helpers

Agent views reuse the same escaping (`esc()`), initial/avatar markup, page header, stat card, and chart card helpers as the customer dashboard, so both apps look and behave identically. The chart adapter honours number formatters so counts render as integers.

# 13 Data & State Management

Because there is no backend, all account and demo state lives in the browser. `assets/js/store.js` is the single module that owns it. The auth pages, the customer dashboard, and the agent dashboard communicate only through its accessors, which means a real API can be wired in later by replacing those accessors.

## 13.1 Public API of StacklyStore

| Method | Purpose |
|---|---|
| `getSession() / setSession(user) / clearSession()` | Read, create, or clear the active session |
| `getUsers() / getUserById(id) / getUserByEmail(email)` | Read user records |
| `createUser(...) / updateUser(id, patch)` | Persist new accounts and profile edits |
| `getDashboardUser(role)` | Resolve the working profile for a role (session, signed-up user, or seeded demo user) |
| `ensureData / getData / saveData / updateData` | Per-user customer demo data helpers |
| `ensureAgentData / getAgentData / saveAgentData / updateAgentData` | Per-agent workspace data helpers |

## 13.2 localStorage keys

| Key | Content |
|---|---|
| `stackly.users` | Array of user records (no passwords or credentials) |
| `stackly.session` | Active session plus per-role sessions under `roles` so one role never blocks the other |
| `stackly.data` | Map of userId to seeded demo data (policies, claims, payments, documents, tickets, notifications, activity) |
| `stackly.agentData` | Map of agentId to agent workspace data |
| `stackly.ui` | Shell preferences such as sidebar collapse |

## 13.3 User record shape

```
{
  "id": "usr_...",
  "fullName": "Maya Rodriguez",
  "email": "maya@example.com",
  "role": "customer",
  "country": "United States",
  "createdAt": "2026-01-14T10:00:00.000Z",
  "policyInterest": "Home",
  "agencyId": "",
  "profile": { "firstName": "", "lastName": "", "address": "", ... },
  "prefs": { "theme": "system", "emailUpdates": true, ... }
}
```

## 13.4 Seeding and safety

- Demo data seeds deterministically per user id (FNV-1a hash + xorshift RNG) on first load, so charts render immediately and stay consistent across reloads.
- User-facing text is escaped with `esc()` before insertion into the DOM; no credentials are stored or logged.
- This is a front-end-only simulation. A production build must enforce authentication and authorization server-side.

# 14 Responsiveness

The site is fully responsive down to 320 px with no cropping or sideways scroll. Responsiveness is achieved with fluid `clamp()` type, a fluid gutter, grid collapses, and a layered set of media queries.

## 14.1 Breakpoints

| Breakpoint | Purpose |
|---|---|
| `1120px` | First tightening: hero grids and catalog grids reduce columns and gaps across the marketing pages |
| `920px` | Two-column layouts collapse to one column (about, products, claims, contact, resources); split carousels stack |
| `680px` | Mobile navigation replaces the desktop nav; stat rows, pricing grids, and side-by-side cards stack; typography drops a step |
| `390px / 360px / 320px` | Shared display-type steps so h1/h2 keep scaling; padding tightens; multi-column strips collapse to single column |

## 14.2 Approach

- **Fluid type.** Headings use `clamp()` with viewport units, so most of the scale is handled without media queries; the small passes only adjust the floor and ceiling.
- **Fluid layout.** The container is `min(1240px, calc(100% - 64px))`, so gutters scale down automatically.
- **Wide tables.** The coverage matrix is wrapped in a scroll container (`.products-matrix-scroll`) so the comparison table never breaks the layout on small screens.
- **Mobile menu.** The menu toggle swaps the desktop nav for a slide-down panel; the page body is locked against scroll while open.
- **Shared pass.** `responsive.css` is loaded last by every inner page and scoped with `body[class]`, giving one consistent type scale and small-screen behaviour everywhere.
- **Defensive rules.** Media (`img, svg, video, canvas`) are capped at 100% width, and long heads use breaking rules so nothing overflows the viewport.

## 14.3 Accessibility within responsive layout

Reordered columns (the auth panel on signup, for example) use `order` rather than abso-positioning so the source order stays meaningful for screen readers. Touch targets on the mobile menu and drawer close buttons remain at least 44 px tall.

# 15 Deployment (GitHub Pages)

Since the project is fully static, GitHub Pages can host it directly from the repository. There is no build step and all asset paths are relative, so the site works from the `/` root of any GitHub Pages site without further configuration.

## 15.1 Prerequisites

- A GitHub account, a local `git` installation, and a repository created for the project.
- GitHub Pages must be enabled for the chosen repository (free for public repositories; a paid plan applies to private ones).

## 15.2 Steps

1. Initialise the repository and stage all files:

```
git init
git add .
git commit -m "Initial commit - Stackly Insurance Service"
```

2. Point the local repository at the remote and push:

```
git remote add origin https://github.com/<user>/<repo>.git
git branch -M main
git push -u origin main
```

3. Open the repository on GitHub, go to *Settings* → *Pages*, set **Source** to **Deploy from a branch**, choose the **main** branch, and select the **root** folder. Then save.
4. GitHub builds the Pages site without any build command and publishes it at:

```
https://<user>.github.io/<repo>/
```

5. Add the repository URL under *Settings* → *Pages* as the custom domain only if a domain is owned. A public custom domain requires a matching DNS or CNAME record.

## 15.3 Notes

- Every future deploy is just a `git push` to `main`; GitHub Pages picks up the new files automatically.
- The site makes no server-side calls, so it works on GitHub Pages exactly as it does locally. The Google Fonts stylesheet and the dashboard chart CDNs are fetched from the browser at runtime.
- Do not commit secrets. This project stores only demo data in `localStorage` and contains no credentials.

# 16 Conclusion

## 16.1 What Was Built

The project ships eleven pages that cover the full insurance journey: six marketing pages that explain the products and the company, a fully validated contact flow, a recoverable 404 page, two focused auth pages, and two complete role-based dashboards. All of it runs on hand-authored HTML, CSS, and vanilla JavaScript with no framework and no build step.

## 16.2 Strengths

- **Consistency.** One tokenized design system drives color, type, spacing, radius, and shadow across every page and both dashboards.
- **Performance.** All imagery is WebP under 100 KB, scripts defer, charts load lazily, and static files load fast and deterministically.
- **Accessibility.** Semantic landmarks, ARIA roles, keyboard operability, focus management, live regions, and `prefers-reduced-motion` support are applied throughout.
- **Separation of concerns.** Shared shell behaviour lives once in `main.js`, data lives once in `store.js`, and page logic is namespaced per page.

## 16.3 Limitations

- The auth and dashboard layers simulate a backend with `localStorage`; there is no real authentication or authorization, so sensitive data must never be trusted to this layer in production.
- Chart-heavy dashboard views score around 90+ on Lighthouse rather than 95+; this is a documented tradeoff of the deliverable.

## 16.4 Recommended Next Steps

1. Replace the `store.js` accessors with API calls and move authentication server-side.
2. Add server-side rendering or a shared include step if the duplicated header/footer markup becomes a maintenance burden.
3. Connect the contact, newsletter, and signup forms to a mail or form service behind the existing validation.
4. Add internationalisation and a structured content source for the marketing pages.

With the current architecture, those next steps are additive: nothing in the design system, page structure, or data layer needs to be thrown away. The project is a coherent, maintainable foundation that can grow from a static showcase into a connected product without a rewrite.